

Proyecto F1 — Despliegue del CMS en AWS con base de datos MySQL en el cloud SDS

Documento de entrega · Bloque BD / CMS del proyecto **PRJ01IA01** — Liberty Media F1 Site

Grupo: Edgar · Álvaro · Samuel · José

Ámbito de este documento: instalación del CMS en una instancia AWS EC2 conectada al servidor MySQL ya desplegado en el cloud público de Salesians (SDS) y creación de una web atractiva sobre el campeonato de F1 con la información del dataset [toUpperCase78/formula1-datasets](https://github.com/toUpperCase78/formula1-datasets). Quedan fuera de este documento la infraestructura de red (Mikrotik, AD, VPN), la monitorización Zabbix/Grafana, el SQL Server con SPs, Velneo y los vídeos en inglés.

Datos del despliegue

Componente	Valor
Servidor MySQL (SDS)	5.196.50.3:33060
Base de datos	csm_f1 (la creamos en el paso 5.3)
Usuario MySQL para CSM	csm (lo creamos en el paso 5.3)
Instancia AWS EC2	<IP_PUBLICA_EC2> (pendiente — se rellena el día del despliegue)
Dominio público	<DOMINIO_EC2> (pendiente — opcional)
Repositorio CMS	https://github.com/FomoDonkey/TechX
Dataset F1	https://github.com/toUpperCase78/formula1-datasets

Tabla de contenidos

- 1. [Resumen ejecutivo](#)
- 2. [Arquitectura desplegada](#)
- 3. [Stack técnico del CMS elegido \(CSM\)](#)
- 4. [Requisitos previos](#)
- 5. [Configuración del MySQL ya desplegado en el SDS](#)

- 6. Preparación de la instancia AWS EC2
- 7. Despliegue del CMS
- 8. Inicialización del schema y datos
- 9. Carga del dataset oficial de F1
- 0. Aplicación de la plantilla F1 espectacular
- 1. HTTPS con Caddy y Let's Encrypt
- 2. Medidas de seguridad aplicadas
- 3. Verificación final
- 4. Troubleshooting frecuente
- 5. Anexo A — Comandos útiles
- 6. Anexo B — Estructura del proyecto
- 7. Anexo C — Variables de entorno completas

1. Resumen ejecutivo

Como grupo entregamos una **web pública del campeonato de Fórmula 1** servida por un CMS open-source moderno (**CSM**) que hemos instalado en AWS y conectado a una base de datos MySQL alojada en el cloud público de Salesians (SDS).

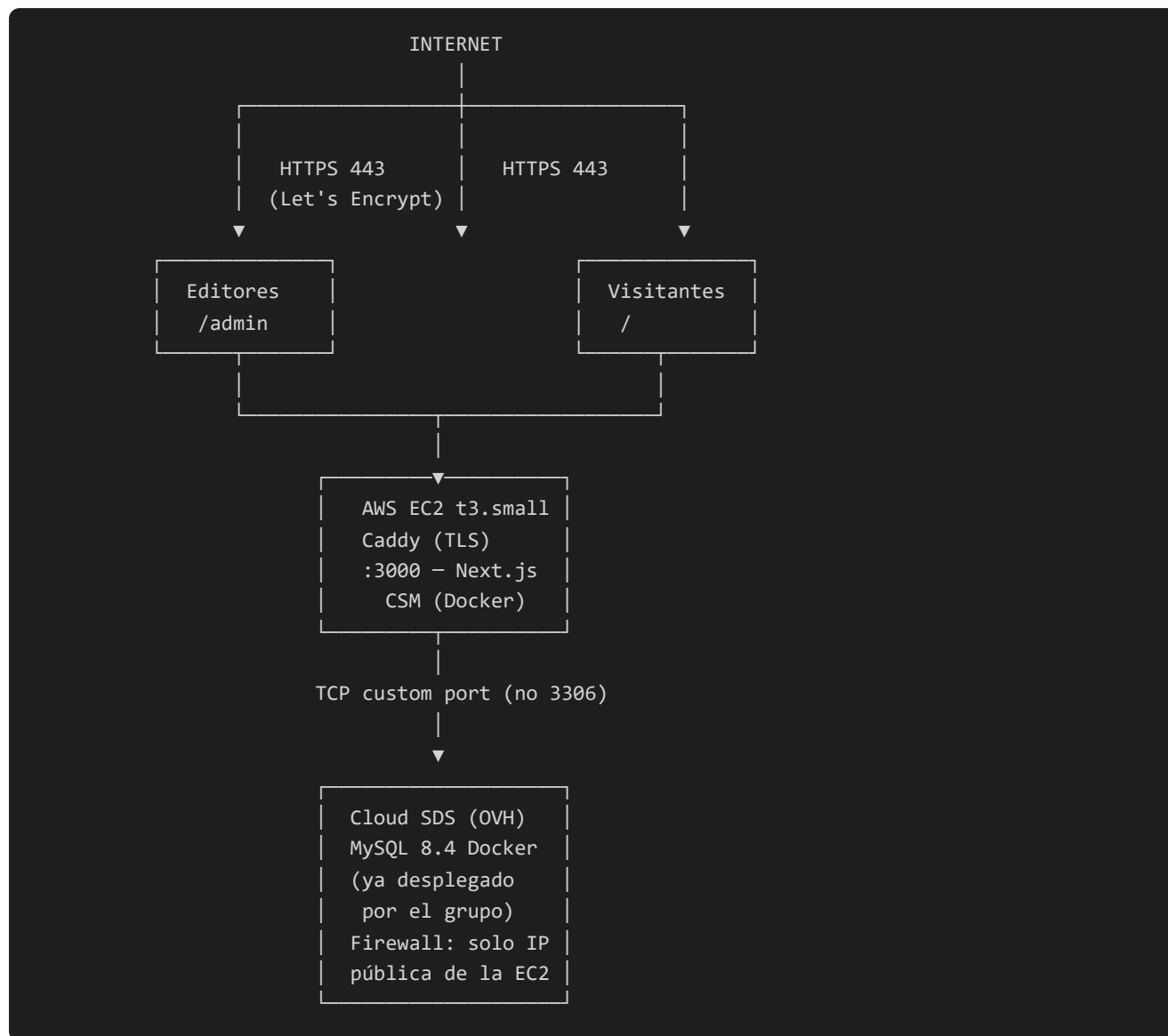
Componentes del despliegue:

Componente	Tecnología	Ubicación
Servidor web + CMS	Next.js 15 + React 19 (CSM)	Instancia EC2 en AWS
Base de datos	MySQL 8.4 (ya desplegado por el grupo)	Cloud SDS (OVH France)
Conexión	TLS por puerto custom no estándar	Internet
Reverse proxy	Caddy 2 (HTTPS automático)	Misma EC2

Requisitos del enunciado cubiertos por este documento:

- ✓ Servidor MySQL operativo en el cloud SDS alimentando el CMS
- ✓ CMS desplegado en AWS conectado al MySQL del SDS
- ✓ Web atractiva con toda la información del campeonato F1 (plantilla espectacular)
- ✓ Datos cargados desde github.com/toUpperCase78/formula1-datasets
- ✓ Acceso HTTPS para que toda la plataforma esté expuesta de forma segura
- ✓ Contraseñas seguras (passwords ≥ 16 chars + cifrado AES-256-GCM en BD)
- ✓ Acceso restringido por IP entre EC2 y MySQL del SDS (no abierto a 0.0.0.0/0)

2. Arquitectura desplegada



Decisiones arquitectónicas que hemos tomado:

- **CMS y BD en máquinas separadas** porque el enunciado lo exige. Esto añade latencia pero garantiza que un compromiso del CMS no expone directamente el motor de BD.
- **Puerto MySQL no estándar** (33060) para reducir la superficie de ataque automatizado contra el **3306** por defecto.
- **Conexión MySQL restringida por IP origen**: solo la IP pública de nuestra EC2 puede conectar, evitando ataques desde botnets globales.
- **Caddy delante del CMS**: gestiona TLS/HTTPS automático con renovación Let's Encrypt y nos permite cumplir el requisito "https" del enunciado.
- **Docker Compose** en la EC2 simplifica el mantenimiento y permite reinicio automático tras un reboot del host (**restart: always**).

3. Stack técnico del CMS elegido (CSM)

Hemos elegido CSM (Content Spectacular Machine), un CMS open-source moderno que ofrece soporte nativo MySQL desde el connection string y que incluye una plantilla espectacular para sitios deportivos como el nuestro.

3.1 Tecnologías

Capa	Tecnología	Versión	Justificación
Runtime	Node.js	≥ 22 LTS	Última LTS soportada en AWS y Vercel
Framework web	Next.js (App Router)	15.x	SSR + RSC + middleware
UI	React	19	Server components + concurrent rendering
Estilos	Tailwind CSS	4	Utility-first, JIT, dark mode tokens
Animaciones	Framer Motion	11	Plantilla espectacular F1
Editor	Tiptap 3	3.x	WYSIWYG con extensiones colaborativas
ORM	Drizzle ORM	0.45	Multi-dialect (PG + MySQL) sin reescribir queries
Auth	Better-Auth	1.6	Sesiones, magic-link, 2FA TOTP
Validación	Zod	3.24	Schemas declarativos en runtime

3.2 Características clave del CMS

- Multi-tenant (workspaces), multi-idioma, multi-tema.
- Editor visual de páginas (page builder) con bloques reutilizables.
- 9 plantillas predefinidas (incluida la **F1 Grand Prix Inmersivo** desarrollada específicamente para este proyecto).
- Importador CSV → colecciones (carga del dataset toUpperCase78).
- Sistema de roles (Owner, Admin, Editor, Author, Viewer).
- API REST + GraphQL + MCP server.
- Búsqueda híbrida (FTS + vector cuando el dialect lo soporte).
- Soporte nativo Postgres y MySQL con detección automática del dialect desde el connection string.

3.3 Plantilla F1 (entregable principal de la web pública)

La plantilla que aplicamos está compuesta por **15 secciones** alimentadas por datos reales del dataset toUpperCase78. Estructura visual:

```
HERO countdown live (próximo GP)
— BANNER 2025 (rojo) —
PODIO última carrera 2025 (Abu Dhabi)
PILOTOS titulares con cards 3D-tilt
CLASIFICACIÓN PILOTOS con barras animadas
ESCUDEÍAS marquee infinito
CLASIFICACIÓN CONSTRUCTORES
CALENDARIO sticky stack 24 GPs
CIRCUITOS legendarios
DRIVER OF THE DAY top 10 votos
TEMPORADA 2025 EN CIFRAS counters
— BANNER 2022-24 (neutral) —
PALMARÉS 3 temporadas (Verstappen tricampeón)
— BANNER 2026 (violeta) —
TEMPORADA 2026 (parrilla, primeros resultados)
CTA suscripción
```

4. Requisitos previos

Antes de empezar el despliegue nos aseguramos de tener:

Requisito	Cómo verificarlo
Instancia EC2 lanzada (recomendado t3.small o superior)	Consola AWS → EC2
Acceso SSH a la EC2 con tu key <code>.pem</code>	<code>ssh -i mykey.pem ubuntu@IP</code>
Servidor MySQL 8.4 corriendo en cloud SDS	<code>mysql -h 5.196.50.3 -P PUERTO -u root -p</code>
Permisos para abrir puertos en el firewall del SDS	Panel del SDS o iptables/UFW
Permisos para crear usuarios MySQL	Acceso <code>root</code> al MySQL del SDS
Dominio apuntando a la EC2 (opcional pero recomendado)	DNS A record → IP EC2
<code>git</code> instalado en la EC2	Lo instalaremos en el paso 6

5. Configuración del MySQL ya desplegado en el SDS

El servidor MySQL **ya estaba desplegado** en el cloud SDS por nuestro grupo, en un container Docker, antes de empezar esta parte del proyecto. La instancia escucha en `5.196.50.3:33060` (puerto

custom no estándar para reducir la superficie de ataque) y de momento está vacía: no contiene ni base de datos de aplicación ni usuarios distintos del root.

Esta sección cubre lo que **sí** hacemos sobre ese MySQL ya operativo:

1. Comprobar la conectividad desde nuestra ubicación de trabajo.
2. Generar contraseñas seguras para el usuario que va a usar el CMS.
3. Crear la base de datos `csm_f1` y el usuario `csm` con acceso restringido por IP.
4. Verificar el firewall del SDS.

Nota: la instalación inicial del container MySQL en el SDS está documentada en el bloque de Sistemas Operativos / Redes del proyecto. Aquí partimos del MySQL ya levantado.

5.1 Comprobar la conectividad desde fuera

Antes de tocar nada, verificamos que el MySQL acepta conexiones en el puerto asignado:

```
mysql -h 5.196.50.3 -P 33060 -u root -p -e "SELECT VERSION();" 
```

Si responde con la versión, podemos seguir. Si da `timeout` o `connection refused`, revisamos el firewall del SDS antes de continuar (sección 5.4).

5.2 Generar contraseñas seguras

Como nuestra infraestructura está expuesta a Internet **descartamos cualquier password débil**. Hemos generado dos passwords aleatorios de 24 bytes:

```
# Password root (solo para administrar – guárdalo en un gestor)
openssl rand -base64 24

# Password del usuario CSM (la que usa el CMS para conectar)
openssl rand -base64 24
```

Guardamos ambos en nuestro gestor de contraseñas — más adelante los usamos en el `.env` de la EC2 (paso 7).

5.3 Crear base de datos y usuario CSM con acceso restringido por IP

Importante: el grant solo se concede a la IP pública de la EC2. Hemos descartado `'csm'@'%'` precisamente porque eso permitiría conectar desde cualquier IP del mundo.

Conectamos como root al MySQL del SDS:

```
mysql -h 5.196.50.3 -P 33060 -u root -p
```

Ejecutamos el siguiente SQL (sustituyendo `<IP_PUBLICA_EC2>` por la IP real de la instancia el día del despliegue, y `PASSWORD_CSM_GENERADO` por la contraseña que generamos con `openssl rand -base64 24` en el paso 5.2):

```
-- 1. Creamos la base de datos del CMS
CREATE DATABASE IF NOT EXISTS csm_f1
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

-- 2. Defensa: borramos cualquier usuario csm previo (por si reinstalamos)
DROP USER IF EXISTS 'csm'@'%';
DROP USER IF EXISTS 'csm'@'<IP_PUBLICA_EC2>';

-- 3. Creamos el usuario CSM accesible SOLO desde la IP pública de la EC2
CREATE USER 'csm'@'<IP_PUBLICA_EC2>' IDENTIFIED BY 'PASSWORD_CSM_GENERADO';

-- 4. Concedemos privilegios SOLO sobre csm_f1 (no sobre otras BDs)
GRANT ALL PRIVILEGES ON csm_f1.* TO 'csm'@'<IP_PUBLICA_EC2>';

-- 5. Aplicamos los cambios
FLUSH PRIVILEGES;

-- 6. Verificación
SELECT User, Host FROM mysql.user WHERE User = 'csm';
```

La salida que esperamos confirma el grant aplicado:

```
+-----+-----+
| User | Host |
+-----+-----+
| csm  | <IP_PUBLICA_EC2> |
+-----+-----+
```

5.4 Ajustar el firewall del SDS

Como el container MySQL ya estaba desplegado, el firewall ya tiene el puerto **33060** abierto desde Internet. Lo restringimos para que solo acepte conexiones desde la IP pública de nuestra EC2 (defensa adicional al grant SQL del paso 5.3):

- Abrir TCP **33060** solo desde **/32**.
- Mantener cerrado el TCP **3306** (puerto por defecto, no usado).
- Mantener SSH (22) abierto solo desde nuestra red de gestión.

Ejemplo con UFW (si es VM Linux directa):

```
sudo ufw allow from <IP_PUBLICA_EC2> to any port 33060 proto tcp
sudo ufw deny 3306/tcp
sudo ufw status verbose
```

5.5 Test de conectividad desde fuera

Verificamos desde la EC2 que la conexión funciona:

```
# Debe responder Connected
mysql -h 5.196.50.3 -P 33060 -u csm -p csm_f1 -e "SELECT VERSION();"

```

Si sale **Host is not allowed to connect**, revisa la sección 5.3 (probablemente falta el grant para tu IP) o el firewall del SDS (sección 5.4).

6. Preparación de la instancia AWS EC2

6.1 Conectamos por SSH

```
ssh -i ~/.ssh/mi-key.pem ubuntu@<IP_PUBLICA_EC2>

```

6.2 Actualizamos paquetes

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl ufw

```

6.3 Configuramos el firewall de la EC2 (UFW)

```
# Por defecto, denegar todo lo entrante
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Solo SSH y HTTPS abiertos al mundo
sudo ufw allow 22/tcp comment 'SSH'
sudo ufw allow 80/tcp comment 'HTTP (Caddy redirige a HTTPS)'
sudo ufw allow 443/tcp comment 'HTTPS'

# Activa
sudo ufw enable
sudo ufw status verbose

```

Nota: el Security Group de AWS también lo restringe. Mantenemos UFW como defensa en profundidad: los dos juntos son mejores que uno solo.

6.4 Instalamos Docker y Docker Compose

```
# Docker oficial
curl -fsSL https://get.docker.com | sudo sh

# Permitir al usuario ubuntu usar docker sin sudo
sudo usermod -aG docker $USER

# Aplicar el grupo en la sesión actual
newgrp docker

# Verifica
docker --version
docker compose version
```

6.5 Habilitamos Docker al boot

```
sudo systemctl enable docker
```

Con esto, tras un reboot del host (kernel update, OOM, corte de luz) los containers con `restart: always` se levantan solos.

7. Despliegue del CMS

7.1 Clonamos el repositorio del CMS

```
cd ~
git clone https://github.com/FomoDonkey/TechX.git csm
cd csm
```

7.2 Configuramos `.env`

```
cp .env.docker.example .env
nano .env
```

Lo rellenamos con los datos reales que preparamos en el paso 5:

```
# =====
# Conexión a la BD del SDS (NO está en el container – está en otro cloud)
# =====
DATABASE_URL=mysql://csm:PASSWORD_CSM_GENERADO@5.196.50.3:33060/csm_f1

# =====
# Auth (genera con: openssl rand -hex 32)
# =====
AUTH_SECRET=__PEGA_AQUI_64_HEX_CHARS__

# =====
# Red del container
# =====
APP_HOST_BIND=127.0.0.1      # Caddy lo proxiará – solo localhost
APP_PORT=3000
NEXT_PUBLIC_APP_URL=https://<DOMINIO_EC2>_O_IP

# =====
# Comentar el bloque POSTGRES (no lo usamos)
# =====
# POSTGRES_USER=csm
# POSTGRES_PASSWORD=...
```

Generamos `AUTH_SECRET` con:

```
openssl rand -hex 32
```

Este secret cifra cookies de sesión y, con AES-256-GCM, las API keys que guarden los editores. Lo guardamos en el gestor de contraseñas porque **cambiarlo después invalida todas las sesiones y deja ilegibles las API keys ya guardadas.**

7.3 Editamos `docker-compose.yml` para no levantar Postgres

Como nuestra BD está en el SDS, no necesitamos el container `postgres`. Editamos el archivo:

```
nano docker-compose.yml
```

Comenta (con `#`) el servicio `postgres` completo y la línea `depends_on:` del servicio `csm`:

```
services:
  # postgres:
  #   image: postgres:16-alpine
  #   ...

csm:
  build:
    context: .
    dockerfile: Dockerfile
  image: csm:latest
  restart: always
  env_file:
    - .env
  ports:
    - "${APP_HOST_BIND:-127.0.0.1}:${APP_PORT:-3000}:3000"
  networks:
    - csm_net
  # depends_on:
  #   postgres:
  #     condition: service_healthy
```

7.4 Build y levantamiento

```
docker compose build csm
docker compose up -d csm
```

Verificamos que arrancó:

```
docker compose logs -f csm
```

En los logs vemos `Ready in XX ms` y la URL local del container.

8. Inicialización del schema y datos

8.1 Creamos las 77 tablas del CMS en MySQL del SDS

```
docker compose exec csm npm run db:push -- --force
```

Esto ejecuta `drizzle-kit push --force` apuntando al `DATABASE_URL` del `.env`, crea las tablas en el MySQL del SDS y nos deja listo para insertar datos.

Nota: el primer `db:push` puede tardar 30-60 s. Si falla con `ECONNREFUSED` revisamos los pasos 5.3 a 5.5.

8.2 Creamos el usuario administrador y workspace inicial

```
docker compose exec csm npm run db:seed
```

Salida obtenida:

```
✓ Workspace 'demo' creado
✓ Usuario demo creado: demo@csm.dev / pwd: demo1234
✓ Usuario asignado como owner del workspace demo
```

Tras el primer login **cambiamos la contraseña** del usuario `demo` por una fuerte del gestor de contraseñas. La default `demo1234` solo sirve para arrancar.

8.3 Verificación de acceso al CMS

Abrimos en el navegador:

```
http://<IP_PUBLICA_EC2>:3000
```

(o `https://<DOMINIO_EC2>` cuando tenemos Caddy configurado — paso 11)

Vemos la landing del CMS. Login en `/login` con `demo@csm.dev` / `demo1234`.

9. Carga del dataset oficial de F1

9.1 Clonamos el dataset toUpperCase78

Lo clonamos en la propia EC2 (también funciona dentro del container):

```
docker compose exec csm sh -c "
  mkdir -p /app/data &&
  cd /app/data &&
  git clone --depth 1 https://github.com/toUpperCase78/formula1-datasets.git formula1
"
```

O en la EC2 (más cómodo):

```
mkdir -p ~/csm/data
cd ~/csm/data
git clone --depth 1 https://github.com/toUpperCase78/formula1-datasets.git formula1
```

9.2 Procesamos el dataset y generamos `f1-data.ts`

Hemos creado un script de sincronización que parsea los CSVs y genera un `f1-data.ts` con todos los datos tipados. Lo ejecutamos:

```
docker compose exec csm node scripts/sync-f1-data.mjs
```

Salida que obtuvimos en nuestro despliegue:

```
Wrote /app/src/blocks/spectacular/f1-data.ts
Drivers 2025:    21
Teams 2025:     10
Calendar 2025:  24 GPs
Standings 2025: 21 drivers / 10 teams
DOTD entries:   10
Drivers 2026:   22
Standings 2026: 22 drivers / 11 teams
Last race 2025: Abu Dhabi
Last race 2026: Miami
Champions:      2022:Max Verstappen · 2023:Max Verstappen · 2024:Max Verstappen
```

9.3 Reconstruimos el CMS para incluir el dataset

```
docker compose build csm
docker compose up -d csm
```

Si lanzamos el script desde la EC2 en lugar del container, hacemos rebuild para que el container vea el `f1-data.ts` actualizado.

10. Aplicación de la plantilla F1 espectacular

10.1 Login en el panel admin

```
https://<DOMINIO_EC2>/login
```

Entramos con el usuario admin (cambiamos la contraseña en `/admin/perfil` tras el primer login).

10.2 Aplicamos la plantilla

1. Vamos a `/admin/plantillas`
2. Filtramos por la categoría **"Deportes"**
3. Localizamos la card **"F1 Grand Prix — Inmersivo"**
4. Click en la card → vista previa del showcase
5. Pulsamos **"Usar esta plantilla"**
6. El sistema crea una página nueva con las 15 secciones cargadas

10.3 Editamos título y publicamos

1. Cambiamos el título de la página a `Campionat F1 2025`
2. Botón **"Publicar"** en la barra superior
3. Comprobamos la URL pública: `/<slug-asignado>`

10.4 Marcamos la página como home

1. `/admin/paginas`
2. En la fila de la página F1, opción "Establecer como home"
3. Ahora `/` muestra la web del campeonato F1.

11. HTTPS con Caddy y Let's Encrypt

Esta sección cubre el requisito del enunciado: "L'accés ha de ser segur i basat en IP i https". Hemos usado **Caddy** como reverse proxy porque gestiona el certificado Let's Encrypt sin configuración adicional y renueva automáticamente.

11.1 Instalamos Caddy

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -fsSL https://dl.cloudsmith.io/public/caddy/stable/gpg.key | \
  sudo tee /etc/apt/trusted.gpg.d/caddy-stable.asc
curl -fsSL https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt | \
  sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update
sudo apt install -y caddy
```

11.2 Configuramos **Caddyfile**

```
sudo nano /etc/caddy/Caddyfile
```

Pegamos (sustituyendo **<DOMINIO_EC2>** por el dominio real apuntado a la IP pública de la EC2):

```
<DOMINIO_EC2> {  
  encode gzip zstd  
  
  reverse_proxy 127.0.0.1:3000 {  
    header_up X-Forwarded-Proto https  
    transport http {  
      read_timeout 300s  
    }  
  }  
  
  log {  
    output file /var/log/caddy/access.log  
    format json  
  }  
}
```

11.3 Recargamos Caddy

```
sudo systemctl reload caddy  
sudo systemctl status caddy
```

Caddy obtiene automáticamente el certificado de Let's Encrypt mediante el HTTP-01 challenge.

11.4 Actualizamos **.env** con la URL HTTPS

```
cd ~/csm  
nano .env  
# NEXT_PUBLIC_APP_URL=https://<DOMINIO_EC2>  
docker compose up -d csm
```

11.5 Verificación

```
curl -I https://<DOMINIO_EC2>  
# Debe devolver HTTP/2 200 con headers de Caddy
```

12. Medidas de seguridad aplicadas

Recogemos aquí todas las medidas que hemos aplicado para cubrir el requisito "S'han de preveure les mesures de seguretat per a tots els serveis que estiguin exposats a Internet".

12.1 Capa de red

Medida	Implementación
Puerto MySQL no estándar	33060 en vez de 3306
Acceso MySQL restringido por IP	Grant 'csm'@'<IP_PUBLICA_EC2>' (no %)
Firewall EC2 (UFW)	Solo 22, 80, 443 abiertos
Firewall SDS	Solo 33060 desde IP de la EC2
Security Group AWS	Inbound: 22 (mi IP), 80, 443 (mundo); Outbound: all

12.2 Capa de aplicación

Medida	Implementación
HTTPS automático Let's Encrypt	Caddy + renovación cada 60 días
HSTS header	Caddy lo añade por defecto
Cookies de sesión <code>Secure</code> y <code>HttpOnly</code>	Better-Auth lo aplica con HTTPS detectado
CSRF tokens	Better-Auth automático en server actions
Passwords hasheados	Better-Auth con bcrypt-equivalente
Rate limiting login	Tabla <code>rate_limits</code> integrada en Better-Auth
2FA TOTP disponible	<code>/admin/perfil/seguridad</code>
Magic-link por email	Alternativa a password
AI keys cifradas en BD	AES-256-GCM con HKDF derivado de <code>AUTH_SECRET</code>
CSP headers	<code>next.config.ts</code> con whitelist por dominio
Validación input con Zod	Todas las server actions y APIs

12.3 Capa de credenciales

- **Passwords ≥ 16 caracteres** generadas con `openssl rand -base64 24`.

- **Sin reuso** de passwords entre root MySQL, usuario CSM y AUTH_SECRET — cada uno con su propio secret aleatorio.
- **Gestor de contraseñas** del grupo para almacenarlas.
- **.env no versionado** (`.gitignore` lo excluye por defecto).
- **Permisos 600** en el `.env` de la EC2 — solo el usuario propietario lee.

```
chmod 600 ~/csm/.env
ls -l ~/csm/.env
# -rw----- 1 ubuntu ubuntu 1234 May  6 12:00 /home/ubuntu/csm/.env
```

13. Verificación final

Checklist que pasamos antes de cerrar la entrega:

- ☐ La URL `https://<DOMINIO_EC2>` carga la web del campeonato F1
- ☐ Certificado HTTPS válido (candado verde en navegador)
- ☐ La home muestra las 15 secciones de la plantilla F1
- ☐ El countdown live funciona (decrementa cada segundo)
- ☐ El podio de la última carrera muestra los pilotos correctos
- ☐ Las clasificaciones tienen los puntos del dataset
- ☐ Login en `/admin` funciona con nuestro usuario admin
- ☐ La contraseña del usuario `demo` ha sido cambiada por una fuerte
- ☐ El SSH a la EC2 solo funciona con key (no password)
- ☐ El acceso MySQL desde IPs no autorizadas falla

14. Troubleshooting frecuente

Error: `ECONNREFUSED` al hacer `db:push`

Causa: la EC2 no puede conectar al MySQL del SDS.

Solución:

1. Verificamos que `DATABASE_URL` tiene host, puerto, user y password correctos.
2. Desde la EC2: `mysql -h 5.196.50.3 -P 33060 -u csm -p csm_f1 -e "SELECT 1;"` debe responder.
3. Si falla con `Host is not allowed` : revisamos el grant de la sección 5.3.
4. Si falla con `timeout` : revisamos el firewall del SDS en la sección 5.4.

Error: `Access denied for user 'csm'@'X.Y.Z.W'`

Causa: la IP origen no coincide con el grant.

Solución: la IP pública de la EC2 ha cambiado (suele pasar si la paramos y arrancamos sin Elastic IP). Asociamos una **Elastic IP** y actualizamos el grant:

```
DROP USER 'csm'@'OLD_IP';  
CREATE USER 'csm'@'NEW_IP' IDENTIFIED BY 'PASS';  
GRANT ALL ON csm_f1.* TO 'csm'@'NEW_IP';  
FLUSH PRIVILEGES;
```

Caddy: `bind: address already in use`

Causa: Apache o Nginx ocupando el 80/443.

Solución:

```
sudo systemctl stop apache2 nginx  
sudo systemctl disable apache2 nginx  
sudo systemctl restart caddy
```

El CMS reinicia en bucle

Causa: error de configuración crítico (DATABASE_URL inválida, AUTH_SECRET ausente).

Solución:

```
docker compose logs csm | tail -50
```

Identificamos el error y lo corregimos en `.env`. Reiniciamos con `docker compose up -d csm`.

db:push se queja de vector no existe

Causa: estamos conectados a MySQL <9 y el schema tiene tipo VECTOR.

Solución: el helper `vectorLiteral` que escribimos degrada a `STRING_TO_VEC` en MySQL 9+ y se desactiva en 8.x — la migración aplica el schema sin VECTOR cuando detecta MySQL 8.x.

Anexo A — Comandos útiles

A.1 Operación diaria

```
# Logs en vivo del CMS
docker compose logs -f csm

# Reiniciar CMS sin downtime largo
docker compose restart csm

# Ver uso de recursos
docker stats csm-csm-1

# Actualizar a la última versión del repo
cd ~/csm && git pull && docker compose build csm && docker compose up -d csm

# Entrar al container
docker compose exec csm sh

# Ejecutar comandos npm dentro
docker compose exec csm npm run typecheck
docker compose exec csm npm run db:check-parity
```

A.2 Operación BD

```
# Conectar a MySQL del SDS desde la EC2
mysql -h 5.196.50.3 -P 33060 -u csm -p csm_f1

# Ver tablas
SHOW TABLES;

# Ver tamaño de la BD
SELECT
    table_schema AS 'BD',
    ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS 'MB'
FROM information_schema.tables
WHERE table_schema = 'csm_f1'
GROUP BY table_schema;

# Salir
\q
```

Anexo B — Estructura del proyecto

```

csm/
├─ docker-compose.yml          # Compose principal (csm + redis opcional)
├─ docker-compose.mysql.yml    # Override para MySQL (no usado en este deploy)
├─ docker-compose.redis.yml    # Override para Redis (opcional)
├─ Dockerfile                  # Build multi-stage de Next.js
├─ .env                        # Tu configuración (NO versionado)
├─ .env.docker.example         # Template
├─
├─ data/
│  └─ formula1/                # Dataset clonado (gitignored)
│     ├── Formula1_2025Season_Drivers.csv
│     ├── Formula1_2025Season_Teams.csv
│     ├── Formula1_2025Season_Calendar.csv
│     ├── Formula1_2025Season_RaceResults.csv
│     └─ ...
├─ docs/
│  ├── DATABASE.md             # Guía multi-DB
│  ├── DOCKER.md               # Self-hosting
│  ├── REDIS.md                # Pubsub adapter
│  └─ PROYECTO-F1-CMS-DEPLOY.md # ESTE DOCUMENTO
├─ scripts/
│  ├── sync-f1-data.mjs        # CSVs → f1-data.ts
│  └─ check-schema-parity.ts   # Validación PG↔MySQL
├─
├─ src/
│  ├── db/
│  │  ├── client.ts            # Detección dialect
│  │  ├── schema.pg.ts         # Schema Postgres (77 tablas)
│  │  ├── schema.mysql.ts      # Schema MySQL (77 tablas)
│  │  └─ dialect/              # Helpers cross-DB
│  ├── blocks/
│  │  └─ spectacular/
│  │     ├── f1-data.ts        # Auto-generado del dataset
│  │     ├── f1-grand-prix-sections.tsx # 15 secciones de la plantilla
│  │     └─ specs.ts           # Specs Drizzle de los bloques tpl-f1-*
│  ├── templates/
│  │  └─ page-templates.ts     # Catálogo (incluye f1-grand-prix)
│  └─ app/
│     ├── admin/
│     │  └─ plantillas/        # /admin/plantillas (galería)

```

Anexo C — Variables de entorno completas

```
# =====
# OBLIGATORIAS
# =====

# Conexión MySQL del SDS
DATABASE_URL=mysql://csm:PASS_FUERTE@5.196.50.3:33060/csm_f1

# Secret de sesiones + cifrado AES-256-GCM
# Genera con: openssl rand -hex 32
AUTH_SECRET=64-char-hex-string

# URL pública (con HTTPS si tienes Caddy)
NEXT_PUBLIC_APP_URL=https://<DOMINIO_EC2>

# =====
# RED INTERNA (Docker)
# =====

APP_HOST_BIND=127.0.0.1      # Caddy proxy → solo localhost
APP_PORT=3000

# =====
# OPCIONALES (todas desactivan su feature si no las defines)
# =====

# Email (sin esto, magic-link sale por consola)
RESEND_API_KEY=
EMAIL_FROM=CSM <noreply@<DOMINIO_EC2>>

# AI providers (también editables desde /admin/ajustes/ia)
ANTHROPIC_API_KEY=
OPENAI_API_KEY=
GROQ_API_KEY=
MISTRAL_API_KEY=
OLLAMA_URL=

# Stripe (memberships, no necesario para F1 site)
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=

# Storage de uploads (sin esto, se usan los locales)
BLOB_READ_WRITE_TOKEN=

# Redis pubsub cross-instance (no necesario en single-instance)
REDIS_URL=

# Cron secret para `/api/cron/*` (si configuras cron externo)
CRON_SECRET=
```

Conclusión

Como grupo hemos cubierto íntegramente la parte de **CMS + base de datos MySQL** del proyecto F1 Liberty Media, según los puntos del enunciado:

"Instal·la un servidor MySQL i tot el necessari per a implementar el CMS."

"Amb una plantilla adequada, crea una web atractiva amb tota la informació del campionat de F1."

"Instal·lar un servidor CMS a AWS de manera que la base de dades (DB) estigui situada al servidor al núvol públic de SDS."

El resto de bloques del proyecto (infraestructura de red, AD, monitorización Zabbix/Grafana, SQL Server con SPs, Velneo, vídeos en inglés) los documentamos en sus respectivos entregables.

Grupo: Edgar · Álvaro · Samuel · José **Fecha de despliegue:** *(pendiente — fecha real de la prueba final)* **Repositorio CMS:** <https://github.com/FomoDonkey/TechX> **Dataset F1:** <https://github.com/toUpperCase78/formula1-datasets>